Experiment No.: 1 Date: _____

Experiment Question

Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples.

Aim

To implement the FIND-S algorithm in Python and use it to find the most specific hypothesis that is consistent with all the positive training examples read from a .CSV file.

Concept

Concept learning means inferring a general rule (hypothesis) from specific training examples. A hypothesis is a conjunction of attribute constraints, where each constraint is a specific value, '?' (any value is acceptable) or '0' (no value is acceptable). FIND-S starts with the most specific hypothesis and generalises it only as far as needed to cover each positive example. It ignores negative examples, so it returns only one hypothesis, the maximally specific one. Its limitations are that it cannot tell whether it has found the only consistent hypothesis and it is sensitive to noisy data.

Dataset Description

The experiment has five sub-programs. Each uses its own small training set stored in a .CSV file (target value Yes / No); the datasets are given with the sub-programs:

1(a) EnjoySport (6 attributes)

1(b) Buy Mango (4 attributes)

1(c) Loan Approval (4 attributes)

1(d) Flu Diagnosis (4 attributes)

1(e) Play Outdoors (4 attributes)

Algorithm

1. Initialise h to the most specific hypothesis <0, 0, ..., 0>.
2. For each positive training example x: for each attribute constraint a_i in h, if a_i is satisfied by x, do nothing; otherwise replace a_i by the next more general constraint satisfied by x (first the value of x, then '?').
3. Ignore every negative example.
4. After all examples have been processed, output h.

Step-by-step Procedure

1. Import the csv module and load the training data from the .CSV file.
2. Initialise h with '0' for every attribute.
3. Loop over the rows; process only the rows whose target value is Yes.
4. Copy the first positive example into h, then change every attribute that differs in later positive examples to '?'.
5. Print h after each example and print the final hypothesis.

Formulae

Generalisation rule: h[j] = x[j] if h[j] = '0'; h[j] stays unchanged if h[j] = x[j]; h[j] = '?' otherwise.
Hypothesis form: h = <attribute 1, attribute 2, ..., attribute n>

Sub-program 1(a): EnjoySport

Dataset Description

4 training examples stored in enjoysport.csv: 6 input attributes and the target attribute **EnjoySport** (Yes / No). FIND-S is applied to find the most specific hypothesis.

Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

Python Program

```
import csv

with open("enjoysport.csv") as f:
    data = list(csv.reader(f))
header, rows = data[0], data[1:]
n = len(header) - 1                    # number of attributes

h = ["0"] * n                          # most specific hypothesis
print("Initial h :", h)

for i, row in enumerate(rows, 1):
    x, label = row[:-1], row[-1]
    if label == "Yes":                 # FIND-S ignores negative examples
        for j in range(n):
            if h[j] == "0":
                h[j] = x[j]            # first positive example
            elif h[j] != x[j]:
                h[j] = "?"             # generalise the attribute
    print(f"After example {i} ({label:3}):", h)

print("Final most specific hypothesis:", h)
```

Fig. 1a.1 Program (exp1a_find_s.py)

Output

```
Initial h : ['0', '0', '0', '0', '0', '0']
After example 1 (Yes): ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']
After example 2 (Yes): ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
After example 3 (No ): ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
After example 4 (Yes): ['Sunny', 'Warm', '?', 'Strong', '?', '?']
Final most specific hypothesis: ['Sunny', 'Warm', '?', 'Strong', '?', '?']
```

Fig. 1a.2 Output of the program

Result / Conclusion

The positive examples differ in Humidity, Water and Forecast, so these attributes become '?'. The negative example (Rainy, Cold, High, Strong, Warm, Change) does not change h. The most specific hypothesis is **<Sunny, Warm, ?, Strong, ?, ?>**: the person enjoys the sport when the sky is sunny, the air is warm and the wind is strong.

Sub-program 1(b): Buy Mango

Dataset Description

5 training examples stored in mango.csv: 4 input attributes and the target attribute **BuyMango** (Yes / No). FIND-S is applied to find the most specific hypothesis.

Colour	Size	Taste	Skin	BuyMango
Yellow	Big	Sweet	Smooth	Yes
Yellow	Small	Sweet	Smooth	Yes
Green	Big	Sour	Rough	No
Yellow	Big	Sweet	Rough	Yes
Red	Small	Sweet	Smooth	No

Python Program

```
import csv

with open("mango.csv") as f:
    data = list(csv.reader(f))
header, rows = data[0], data[1:]
n = len(header) - 1                    # number of attributes

h = ["0"] * n                          # most specific hypothesis
print("Initial h :", h)

for i, row in enumerate(rows, 1):
    x, label = row[:-1], row[-1]
    if label == "Yes":                 # FIND-S ignores negative examples
        for j in range(n):
            if h[j] == "0":
                h[j] = x[j]            # first positive example
            elif h[j] != x[j]:
                h[j] = "?"             # generalise the attribute
    print(f"After example {i} ({label:3}):", h)

print("Final most specific hypothesis:", h)
```

Fig. 1b.1 Program (exp1b_find_s.py)

Output

```
Initial h : ['0', '0', '0', '0']
After example 1 (Yes): ['Yellow', 'Big', 'Sweet', 'Smooth']
After example 2 (Yes): ['Yellow', '?', 'Sweet', 'Smooth']
After example 3 (No ): ['Yellow', '?', 'Sweet', 'Smooth']
After example 4 (Yes): ['Yellow', '?', 'Sweet', '?']
After example 5 (No ): ['Yellow', '?', 'Sweet', '?']
Final most specific hypothesis: ['Yellow', '?', 'Sweet', '?']
```

Fig. 1b.2 Output of the program

Result / Conclusion

The positive examples differ in Size and Skin, so these become '?'. The two negative examples are ignored. The most specific hypothesis is **<Yellow, ?, Sweet, ?>**: a mango is worth buying when it is yellow and sweet.

Sub-program 1(c): Loan Approval

Dataset Description

5 training examples stored in loan.csv: 4 input attributes and the target attribute **Approved** (Yes / No). FIND-S is applied to find the most specific hypothesis.

Income	Credit	Job	Age	Approved
High	Good	Yes	Young	Yes
Medium	Good	Yes	Old	Yes
Low	Poor	Yes	Young	No
High	Good	No	Old	No
Low	Good	Yes	Old	Yes

Python Program

```
import csv

with open("loan.csv") as f:
    data = list(csv.reader(f))
header, rows = data[0], data[1:]
n = len(header) - 1                    # number of attributes

h = ["0"] * n                          # most specific hypothesis
print("Initial h :", h)

for i, row in enumerate(rows, 1):
    x, label = row[:-1], row[-1]
    if label == "Yes":                 # FIND-S ignores negative examples
        for j in range(n):
            if h[j] == "0":
                h[j] = x[j]            # first positive example
            elif h[j] != x[j]:
                h[j] = "?"             # generalise the attribute
    print(f"After example {i} ({label:3}):", h)

print("Final most specific hypothesis:", h)
```

Fig. 1c.1 Program (exp1c_find_s.py)

Output

```
Initial h : ['0', '0', '0', '0']
After example 1 (Yes): ['High', 'Good', 'Yes', 'Young']
After example 2 (Yes): ['?', 'Good', 'Yes', '?']
After example 3 (No ): ['?', 'Good', 'Yes', '?']
After example 4 (No ): ['?', 'Good', 'Yes', '?']
After example 5 (Yes): ['?', 'Good', 'Yes', '?']
Final most specific hypothesis: ['?', 'Good', 'Yes', '?']
```

Fig. 1c.2 Output of the program

Result / Conclusion

The positive examples differ in Income and Age, so these become '?'. The negative examples are ignored. The most specific hypothesis is **<?, Good, Yes, ?>**: a loan is approved when the applicant has a good credit record and a job.

Sub-program 1(d): Flu Diagnosis

Dataset Description

5 training examples stored in flu.csv: 4 input attributes and the target attribute **Flu** (Yes / No). FIND-S is applied to find the most specific hypothesis.

Fever	Cough	Fatigue	Headache	Flu
Yes	Yes	Yes	Yes	Yes
Yes	Yes	No	Yes	Yes
No	Yes	Yes	No	No
Yes	No	Yes	Yes	No
Yes	Yes	Yes	No	Yes

Python Program

```
import csv

with open("flu.csv") as f:
    data = list(csv.reader(f))
header, rows = data[0], data[1:]
n = len(header) - 1                    # number of attributes

h = ["0"] * n                          # most specific hypothesis
print("Initial h :", h)

for i, row in enumerate(rows, 1):
    x, label = row[:-1], row[-1]
    if label == "Yes":                 # FIND-S ignores negative examples
        for j in range(n):
            if h[j] == "0":
                h[j] = x[j]            # first positive example
            elif h[j] != x[j]:
                h[j] = "?"             # generalise the attribute
    print(f"After example {i} ({label:3}):", h)

print("Final most specific hypothesis:", h)
```

Fig. 1d.1 Program (exp1d_find_s.py)

Output

```
Initial h : ['0', '0', '0', '0']
After example 1 (Yes): ['Yes', 'Yes', 'Yes', 'Yes']
After example 2 (Yes): ['Yes', 'Yes', '?', 'Yes']
After example 3 (No ): ['Yes', 'Yes', '?', 'Yes']
After example 4 (No ): ['Yes', 'Yes', '?', 'Yes']
After example 5 (Yes): ['Yes', 'Yes', '?', '?']
Final most specific hypothesis: ['Yes', 'Yes', '?', '?']
```

Fig. 1d.2 Output of the program

Result / Conclusion

The positive examples differ in Fatigue and Headache, so these become '?'. The negative examples are ignored. The most specific hypothesis is **<Yes, Yes, ?, ?>**: flu is predicted when the patient has both fever and cough.

Sub-program 1(e): Play Outdoors

Dataset Description

5 training examples stored in play.csv: 4 input attributes and the target attribute **Play** (Yes / No). FIND-S is applied to find the most specific hypothesis.

Outlook	Temp	Humidity	Wind	Play
Overcast	Hot	High	Weak	Yes
Overcast	Mild	Normal	Weak	Yes
Sunny	Hot	High	Weak	No
Overcast	Cool	Normal	Strong	No
Overcast	Mild	High	Weak	Yes

Python Program

```
import csv

with open("play.csv") as f:
    data = list(csv.reader(f))
header, rows = data[0], data[1:]
n = len(header) - 1                    # number of attributes

h = ["0"] * n                          # most specific hypothesis
print("Initial h :", h)

for i, row in enumerate(rows, 1):
    x, label = row[:-1], row[-1]
    if label == "Yes":                 # FIND-S ignores negative examples
        for j in range(n):
            if h[j] == "0":
                h[j] = x[j]            # first positive example
            elif h[j] != x[j]:
                h[j] = "?"             # generalise the attribute
    print(f"After example {i} ({label:3}):", h)

print("Final most specific hypothesis:", h)
```

Fig. 1e.1 Program (exp1e_find_s.py)

Output

```
Initial h : ['0', '0', '0', '0']
After example 1 (Yes): ['Overcast', 'Hot', 'High', 'Weak']
After example 2 (Yes): ['Overcast', '?', '?', 'Weak']
After example 3 (No ): ['Overcast', '?', '?', 'Weak']
After example 4 (No ): ['Overcast', '?', '?', 'Weak']
After example 5 (Yes): ['Overcast', '?', '?', 'Weak']
Final most specific hypothesis: ['Overcast', '?', '?', 'Weak']
```

Fig. 1e.2 Output of the program

Result / Conclusion

The positive examples differ in Temp and Humidity, so these become '?'. The negative examples are ignored. The most specific hypothesis is **<Overcast, ?, ?, Weak>**: the game is played when the sky is overcast and the wind is weak.